



PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Software Test Plan (STP)

AttendEase – Student Attendance Management System Team 3 | Course UE24CS341A | PES University

Revision History

Version	Date	Description	Author(s)
1.0	03 October 2026	Initial Software Test Plan for AttendEase.	Team 3

Approvals

Name	USN	Date
Arundhati K	PES1UG24CS083	03-Oct-2026
Anshdeep Singh Sachdeva	PES1UG24CS070	03-Oct-2026
Atharva Ashish Vyas	PES1UG24CS093	03-Oct-2026
Bhoomika Dayanand Jituri	PES1UG25CS807	03-Oct-2026

1. Introduction

1.1 Purpose

This Software Test Plan (STP) defines the scope, approach, resources, and schedule of testing activities for AttendEase. It specifies which features will be tested, the testing levels and types to be applied, the environment and tools required, and the traceability between test cases and the requirements defined in the AttendEase Software Requirements Specification (SRS).

1.2 Scope

This plan covers functional testing of all 38 functional requirements (FR-001–FR-038), non-functional testing of performance, safety, and quality-attribute requirements (NFR-001–NFR-008), and security validation of SEC-001–SEC-006 and the Security Objectives defined in SRS Section 6.3. It applies to the AttendEase web application across its four team-owned feature areas.

1.3 References

- AttendEase Software Requirements Specification (SRS), v1.1, Team 3.
- AttendEase Software Architecture & Design Specification (SAD), v1.0, Team 3.
- PES University SE Level 3 Mini-Project Guidelines and Deliverables Checklist.

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
STP	Software Test Plan
RTM	Requirements Traceability Matrix
FR / NFR	Functional Requirement / Non-Functional Requirement
SEC	Security Requirement (as numbered in the SRS)
CI	Continuous Integration

2. Test Items

- Authentication & Role-Based Access module (Role 1).
- Student Attendance Dashboard (Role 1).
- Student & Academic Management module (Role 2).
- Attendance Recording module (Role 2).
- Attendance Management & Correction Requests module (Role 3).
- Attendance Shortage Identification & Reports module (Role 4).
- REST API layer (all endpoints listed in the SAD, Section 4.3).
- Database schema and data-integrity constraints.

3. Features to be Tested

All functional requirements FR-001 through FR-038, all non-functional requirements NFR-001 through NFR-008, and all security requirements SEC-001 through SEC-006, as defined in the SRS, are in scope for testing. Representative coverage is detailed in the Test Case table in Section 13.

4. Features Not to be Tested

- Third-party infrastructure internals (e.g., the SMTP provider's own delivery guarantees, PostgreSQL's internal engine correctness) — these are assumed reliable and out of scope.
- Load testing beyond the expected single-department scale (not required for the mini-project scope; noted as a future-work item).
- Browser-compatibility testing beyond the latest stable versions of Chrome and Firefox.

5. Test Approach / Strategy

5.1 Security Validation

Security validation is performed as a dedicated testing activity, distinct from functional testing, covering both the Security Objectives and Security Requirements defined in SRS Section 6.3:

- Verify password hashing is applied (no plaintext passwords in the database) and that login endpoints reject invalid credentials correctly.
- Verify role-based access control: attempt to call admin-only and faculty-only endpoints with a student token and confirm 403 Forbidden responses.
- Verify that a student cannot view or modify another student's attendance records via direct API calls (authorization bypass / IDOR check).
- Verify HTTPS is enforced in the staging/production configuration and that session tokens are not exposed in URLs or logs.
- Verify audit fields (acted_by, acted_at) are correctly populated on correction-request approvals/rejections (non-repudiation check).
- Basic input-fuzzing of form and API fields (SQL-injection and XSS payloads) against the registration, attendance, and correction-request endpoints.

5.2 Test Levels

Level	Description	Owner
Unit Testing	Individual service-layer functions tested in isolation (e.g., duplicate-attendance validation logic).	Module owner (per role)
Integration Testing	API endpoint tested against the service and database layers together.	Module owner + reviewer
System Testing	End-to-end workflows across UI → API → DB (e.g., full correction-request lifecycle).	Whole team
Acceptance Testing	Verification against SRS requirements prior to each sprint demo / final submission.	Whole team

5.3 Test Types

- Functional testing — validates each FR behaves as specified.
- Regression testing — re-run automatically via CI on every pull request to catch breakage from new changes.
- Performance testing — validates NFR-001/NFR-002 response-time targets under expected concurrent load.

- Usability testing — informal walkthroughs of the student/faculty UI with team members acting as test users.

5.4 Entry and Exit Criteria

Entry criteria:

- The feature's code has been merged to the main branch via a reviewed pull request and passes CI lint/build checks.

Exit criteria:

- All planned test cases for the feature have been executed with no open critical or high-severity defects.
- Test results have been recorded in the project's test-tracking sheet / GitHub Projects board.

6. Test Environment

Category	Detail
Hardware	Standard developer laptops; no specialized hardware required.
Software	Python 3.11, Django 4.x, PostgreSQL 14+, Docker, modern Chrome/Firefox browsers.
Tools	Django's built-in test framework / pytest-django, Postman or DRF's browsable API for manual API testing, GitHub Actions for CI execution.
Test Data	Seed fixtures representing at least 1 admin, 2 faculty, and 10 student accounts across 2 courses/sections.

7. Test Schedule

Milestone	Target Timeframe	Activity
Unit test scaffolding	Sprint 2	Each role sets up unit tests for their service layer as features are built.
Integration testing	Sprint 3–4	API-level tests added as endpoints stabilize.
System & security testing	Sprint 5	End-to-end and security validation across completed modules.
Acceptance / final regression	Sprint 6	Full regression pass before final submission and demo.

8. Test Deliverables

- This Software Test Plan document.
- Automated unit/integration test suite in the project repository, executed via CI.
- Requirements Traceability Matrix (Section 13).
- Test execution summary / defect log maintained in GitHub Projects.

9. Roles and Responsibilities

Role	Responsibility	Owner(s)
QA Lead	Coordinates the overall test plan, tracks coverage, and owns Sections 5.1 and 13 of this document.	Bhoomika Dayanand Jituri (Testing, CI/CD, Jira & documentation)
Test Engineer (per module)	Writes and executes unit/integration tests for their owned feature area.	Each team member, for their own role's modules
Developer	Fixes defects identified during testing; participates in code review.	All team members
Product Owner / Reviewer	Confirms acceptance criteria are met before sign-off.	Whole team, each sprint review

10. Risks and Mitigation

Risk	Mitigation
Limited time for dedicated security testing within the academic schedule.	Security validation (Section 5.1) is scheduled as its own milestone in Sprint 5, not left to the end.
Inconsistent test coverage across the four independently-owned modules.	QA Lead reviews coverage weekly against the RTM in Section 13.
Test data drift between developer environments.	Shared seed-fixture script checked into the repository and run via docker-compose.

11. Assumptions and Dependencies

- All four team members have local Docker and PostgreSQL environments matching the SAD's technology stack.
- The CI pipeline (GitHub Actions) is configured and operational from Sprint 2 onward.
- The SRS and SAD remain the authoritative source of requirements; any requirement change is reflected in the RTM.

12. Suspension and Resumption Criteria

Testing of a feature is suspended if a blocking defect prevents meaningful further test execution (e.g., the CI build is broken, or the database migrations fail). Testing resumes once the blocking defect is fixed and merged, and the CI pipeline passes again.

13. Test Case Management and Traceability

Representative test cases below (12, exceeding the minimum of 7–10 required) are traceable to SRS requirement IDs and cover functional (FR), non-functional (NFR), and security (SEC) requirements. The full test suite, maintained in the repository, expands this table with complete step-by-step cases for every FR/NFR/SEC.

Test Case ID	Description	Requirement(s)	Expected Result
TC-01	Register a new student account with valid details.	FR-001	Account created; confirmation shown; password stored hashed.
TC-02	Login with an incorrect password three times.	FR-003, SEC-002	Access denied each time; no account/session leaked in

Test Case ID	Description	Requirement(s)	Expected Result
			error message.
TC-03	Admin assigns a faculty member to a course-section.	FR-006, FR-013	Assignment saved; faculty sees the section on their dashboard.
TC-04	Faculty starts an attendance session and marks all students present.	FR-014, FR-016	Session and attendance records created; roster reflects marked status.
TC-05	Faculty attempts to mark attendance twice for the same session/student.	FR-037	Second attempt rejected with a duplicate-record error (409).
TC-06	Student views their attendance dashboard.	FR-020, FR-021	Correct attendance percentage and per-course breakdown displayed.
TC-07	Student raises a correction request for a disputed record.	FR-024, FR-025	Request created with Pending status; faculty notified by email.
TC-08	Faculty approves a pending correction request.	FR-027	Attendance record updated; request marked Approved; audit fields populated.
TC-09	Student account attempts to call an admin-only academic-setup endpoint.	SEC-003, SEC-004	403 Forbidden returned; action logged.
TC-10	Generate a defaulter report for a section below the shortage threshold.	FR-029, FR-031	Report lists exactly the students below threshold with correct percentages.
TC-11	Load the attendance-marking roster for a section of 80 students and measure response time.	NFR-001, NFR-002	Roster loads within 3 seconds; API response within 2 seconds under expected concurrent load.
TC-12	Run the service-layer automated test suite and measure statement/branch coverage.	NFR-006	Automated test coverage on service-layer business logic is at least 70%.

14. Test Metrics and Reporting

- Requirement coverage: percentage of FR/NFR/SEC IDs with at least one passing test case, tracked weekly.
- Defect density: number of defects found per module, tracked via GitHub Projects issue labels.
- CI pass rate: percentage of pull requests passing all automated checks on first submission.
- A summary of these metrics is reviewed by the whole team at each sprint review.

15. Approvals

Name	USN	Date
Arundhati K	PES1UG24CS083	03-Oct-2026
Anshdeep Singh Sachdeva	PES1UG24CS070	03-Oct-2026

Name	USN	Date
Atharva Ashish Vyas	PES1UG24CS093	03-Oct-2026
Bhoomika Dayanand Jituri	PES1UG25CS807	03-Oct-2026